جامعة اللاذقية
كلية الهندسة المعلوماتية
قسم البرمجيات ونظم المعلومات
السنة الخامسة - قواعد معطيات متقدمة

Database Administrator Guidelines for Password Security

Dr.-Ing. Basel Hasan

2025 / 2026

Agenda

- System Variables
- Password Management
  - expiration, reuse and verification-required policies
  - Failed-Login Tracking and Temporary Account Locking
- Server Handling of Expired Passwords
- Password Validation

System Variables

- The MySQL server maintains system variables that configure its operation.
  - A system variable can have a **global value** that affects server operation as a whole, a **session value** that affects the current session.
- **`SET GLOBAL`**

  the changes affect only the currently running server instance**!**
- **`SET PERSIST`**

  the changes affect the currently running server instance and persist across server startups. System variable settings are persisted to a file named **mysqld-auto.cnf** in the data directory*. (see slide 6)

  * DATADIR\mysqld-auto.cnf

  default, the DATADIR is set to C:/ProgramData/MySQL/MySQL Server 8.0/Data

Persisted Settings

- Persisted settings are made at runtime.
- Persisted settings are permanent. They apply across server restarts.
- To persist system variables, no need to have access to file system (e.g. configuration files).
- An administrator with sufficient privileges can reconfigure a server by persisting system variables.
  - More regarding privileges, later!

Example

- The system variable `MAX_CONNECTIONS` determines the number of connections which MySQL will accept. The default value is 151 connections.
- Situation: Our application grows in the market, and we need to increase the number of connections to 200.
  - `SET GLOBAL MAX_CONNECTIONS = 200;`

    vs.

  - `SET PERSIST MAX_CONNECTIONS = 200;`

What is the difference?

Alternatively, add

```
[mysqld]
 MAX_CONNECTIONS = 200
```

to the configuration file `my.ini`*, and (re)start the server.

!

* **C:\ProgramData\MySQL\MySQL Server 8.0**my.ini

mysqld-auto.cnf

- The `mysqld-auto.cnf` file uses a `JSON` format (Java Script Opject Notation).

```
{…,
  "max_connections" : {
     "Value" : "200" ,
     "Metadata" : {
         "Timestamp" : 1709129492535220 ,
         "User" : "root" ,
         "Host" : "localhost"
      }
   },
…}
```

timestamp is in **microseconds (1/1,000,000 second)**

Password Management

- Password expiration, to require passwords to be changed periodically.
- Password reuse restrictions, to prevent old passwords from being chosen again.
- Password verification, to require that password changes also specify the current password to be replaced.
- Dual passwords, to enable clients to connect using either a primary or secondary password.
- Password strength assessment, to require strong passwords. (Password Complexity)
- Password failure tracking, to enable temporary account locking after too many consecutive incorrect-password login failures.

Password Policy

- Password policy implementation should be the first defense for any organization desiring to lower the risk of compromised user passwords.
- Policies can be defined within the database system, or they can be written as a formal agreement of acceptable computer use between management and employees.
- Database server password policies that are configured within the database are enforced throughout the database management system.
- Users are the weakest component within any security architecture.

Password Expiration Policy - global

- Password expiration according to policy is automatic and is based on password age.
  - **`SET GLOBAL default_password_lifetime = 180;`**
  - or, persist the system variables to the `mysqld-auto.cnf` file.
  - **`SET PERSIST default_password_lifetime = 180;`**
  - **`RESET PERSIST IF EXISTS default_password_lifetime;`** `// remove a specific persisted variables`
  - **`RESET PERSIST;`** `// remove all persisted variables`
  - **`SHOW GLOBAL VARIABLES LIKE "%life%";`** `// display a specific variable`

Password Expiration Policy - specific

- The **global** password-expiration policy applies to all accounts that have not been set to override it.
- To establish policy for individual accounts, use the `PASSWORD EXPIRE` option:
  - Require the password to be changed every 90 days
    ```
    ALTER USER 'testUser'@'localhost' PASSWORD EXPIRE
    INTERVAL 90 DAY;
    ```
  - Disable password expiration
    ```
    ALTER USER 'testUser'@'localhost' PASSWORD EXPIRE
    NEVER;
    ```
  - Defer to the global expiration policy
    ```
    ALTER USER 'testUser'@'localhost' PASSWORD EXPIRE
    DEFAULT;
    ```

Server Handling of Expired Passwords

- MySQL provides password-expiration capability, which enables database administrators to require that users reset their password.
- For each connection that uses an account with an expired password, the server either disconnects the client or restricts the client to “sandbox mode”, in which the server permits the client to perform only those operations necessary to reset the expired password. This can be applied using the following system variables:
  - The server disconnects the client `disconnect_on_expired_password=ON`
  - the server puts the client in sandbox mode `disconnect_on_expired_password=OFF`

Expire Password Manually

- To expire an account password manually, use the ALTER USER statement:
  - `ALTER USER 'testUser'@'localhost' PASSWORD EXPIRE;`

```
Command Prompt - mysql -u testUser -p
C:\>mysql -u testUser -p
Enter password: *********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 24
Server version: 8.0.26

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input
statement.

mysql> use mydb;
No connection. Trying to reconnect...
Connection id:    25
Current database: *** NONE ***

ERROR 1820 (HY000): You must reset your password using ALTER USER st
atement before executing this statement.
mysql> ALTER USER 'testUser'@'localhost' IDENTIFIED BY 'aDB2021q1!';

Query OK, 0 rows affected (0.01 sec)

mysql> use mydb;
Database changed
mysql>
```

! If the client can handle expired passwords (e.g. mysql client), the value of `disconnect_on_expired_password` is irrelevant. The server permits the client to connect but puts it in sandbox mode.

Password-reuse Policy

- Applying restrictions to be placed on reuse of **previous** passwords.
- Reuse restrictions can be established based on number of password changes, time elapsed, or both.
- Reuse policy can be established **globally**, and individual accounts can be set to either defer to the global policy or override the global policy with **specific per-account** behavior.
- The password history for an account consists of passwords that have been assigned in the past. New passwords can be restricted from being chosen from this history.

Password-reuse Policy - global

- The system variables `password_history` and `password_reuse_interval` can be used to establish password-reuse policy globally. e.g.
  - add the following to the configuration file `my.ini`, and start the server.

```
[mysqld]
password_history=6
password_reuse_interval=365
```

  - At runtime:

```
SET PERSIST password_history = 6;
SET PERSIST password_reuse_interval = 365;
```

Password-reuse Policy - specific

- To establish policy for individual accounts, the `PASSWORD HISTORY` and `PASSWORD REUSE INTERVAL` options of the `CREATE USER` and `ALTER USER` statements are used. e.g.
  - `ALTER USER 'testUser'@'localhost' PASSWORD HISTORY 5;`
  - `ALTER USER 'testUser'@'localhost' PASSWORD REUSE INTERVAL 365 DAY;`
  - `ALTER USER 'testUser'@'localhost' PASSWORD HISTORY 5 PASSWORD REUSE INTERVAL 365 DAY;`
  - Defer to the global policy for both types of reuse restriction :

```
ALTER USER 'testUser'@'localhost'
  PASSWORD HISTORY DEFAULT
  PASSWORD REUSE INTERVAL DEFAULT;
```

Password_reuse_history	Password_reuse_time
NULL	NULL
NULL	NULL
NULL	NULL
NULL	NULL
NULL	NULL
NULL	NULL

Password Verification-Required Policy

- To require that attempts to change an account password be verified by **specifying the current password** to be replaced.
- This enables DBAs to prevent users from changing a password without proving that they know the current password.
- Possible Risk:
  - A user walks away from a terminal session temporarily without logging out, a malicious user uses the session to change the original user's password. This leads to unfortunate consequences:
    - The original user becomes unable to access database until the account password is reset by an administrator.
    - Until the password reset occurs, the malicious user can access database with the user's changed credentials.

Password Verification-Required Policy - global

- Password-verification policy can be established **globally**, and individual accounts can be set to either defer to the global policy or override the global policy with **specific per-account** behavior.
- The system variables `password_require_current` can be used to establish password-verification policy globally. e.g.
  - add the following to the configuration file `my.ini`, and start the server.
    ```
    [mysqld]
    password_require_current=ON
    ```
  - at runtime:
    ```
    SET PERSIST password_require_current = ON;
    SET PERSIST password_require_current = OFF;
    ```

Password Verification-Required Policy - specific

- To establish policy for individual accounts, the `PASSWORD REQUIRE` option of the `CREATE USER` and `ALTER USER` statements is used. e.g
  - `ALTER USER 'testUser'@'localhost' PASSWORD REQUIRE CURRENT;`
  - `ALTER USER 'testUser '@'localhost' PASSWORD REQUIRE CURRENT OPTIONAL;`
  - Defer to the global policy for both types of reuse restriction:
    ```
    ALTER USER 'testUser'@'localhost'
      PASSWORD REQUIRE CURRENT DEFAULT;
    ```

Password_require_current
NULL
NULL
NULL
NULL
NULL
NULL

.

Password Verification-Required Policy

- A `REPLACE` clause specifies the current password to be replaced:
  - change the current user's password
    ```
    ALTER USER USER() IDENTIFIED BY 'new_Password'
    REPLACE 'current_Password';
    ```
  - change a named user's password
    ```
    ALTER USER 'testUser'@'localhost' IDENTIFIED BY
    'new_Password' REPLACE 'current_Password';
    ```

Dual Password Support

- User accounts are permitted to have dual passwords, designated as **primary** and **secondary** passwords.
- You can use this feature in scenarios where you've multiple applications connected to a database with the same credentials, but you need to change the password.
- Because this change can take seconds to hours depending on the system design, it can lead to consequences poor user experience, downtime, etc.

✓ Using dual passwords is a method to seamlessly change MySQL passwords without locking out users who use the old credentials.

Dual Password Support - workflow

- Situation: An account named `'testUser'@'localhost'` is used by applications to connect to database server, and that the account password is to be changed from 'password_a' to 'password_b'.
- Password Change Workflow:
  1. Change password keeping the old one.
     ```
     ALTER USER 'testUser'@'localhost' IDENTIFIED BY
     'password_b' RETAIN CURRENT PASSWORD;
     ```
  2. Change the applications' passwords.
  3. After all applications have been migrated from the secondary password to the primary password, the secondary password are no longer needed and can be discarded.
     ```
     ALTER USER 'testUser'@'localhost' DISCARD OLD
     PASSWORD;
     ```

Failed-Login Tracking and Temporary Account Locking 1/2

- Administrators can configure user accounts such that too many consecutive login failures cause temporary account locking.
- “Login failure” in this context means failure of the client to provide a correct password during a connection attempt.
- The required **number of login failures** and the **lock time** are configurable per account.
- The options `FAILED_LOGIN_ATTEMPTS` and `PASSWORD_LOCK_TIME` of the CREATE USER and ALTER USER statements can be used:
  -
```
ALTER USER 'testUser'@'localhost'
  FAILED_LOGIN_ATTEMPTS 4
  PASSWORD_LOCK_TIME UNBOUNDED;
```

Failed-Login Tracking and Temporary Account Locking 2/2

- `FAILED_LOGIN_ATTEMPTS` **`N`**
  `N` specifies the number of consecutive incorrect passwords cause temporary account locking.

- `PASSWORD_LOCK_TIME {`**`N`** `|` **`UNBOUNDED`**`}`
  `N` specifies the number of days the account remains locked, or **`UNBOUNDED`** specifies that an account enters the temporarily locked state, the duration of that state is unbounded and does not end until the account is unlocked.

❖ **`N`** can be set in range `0` to `32767`. A value of `0` disables the option.

❖ Any successful login that occurs prior to reaching the `FAILED_LOGIN_ATTEMPTS` value causes failure counting to reset.

Account-unlock methods

- Execution of an `ALTER USER` statement for the account that sets either `FAILED_LOGIN_ATTEMPTS` or `PASSWORD_LOCK_TIME` (or both) to any value (including the current option value).
- An account can be manually locked/unlocked:

```
ALTER USER 'testUser'@'localhost' ACCOUNT LOCK;
ALTER USER 'testUser'@'localhost' ACCOUNT UNLOCK;
```

- Check account state `USER` table:

```
select user, account_locked, User_attributes from mysql.user
where user = 'testUser';
```

	user	account_locked	User_attributes
▶	testUser	N	{"Password_locking": {"failed_login_attempts": 2, "password_lock_time_days": -1}}

- The question is: Who should be able to perform the `ALTER USER` statement?

Extending Database Server Capabilities

- MySQL Server includes a component-based infrastructure for extending server capabilities.
- A component provides services that are available to the server and other components.
- Components must be loaded into the server before they can be used.
- To load/unload a component, `INSTALL | UNINSTALL COMPONENT` statements is used.
- A loader service handles component loading and unloading, and also registers loaded components in the `mysql.component` system table.

INSTALL COMPONENT

- Syntax:
  `INSTALL COMPONENT component_name [, component_name ] ...`

- Example:
  `INSTALL COMPONENT 'file://component1', 'file://component2';`

  URN – Unified Resource Name

- A component is named using a URN that begins with file:// and indicates the base name of the library file that implements the component, located in the directory named by the `plugin_dir` system variable.

```
mysql> show variables like "plugin_dir";
+---------------+-----------------------------------------------------+
| Variable_name | Value                                               |
+---------------+-----------------------------------------------------+
| plugin_dir    | C:\Program Files\MySQL\MySQL Server 8.0\lib\plugin\ |
+---------------+-----------------------------------------------------+
1 row in set (0.00 sec)
```

Password Validation

- The `validate_password` component serves to improve security by enabling strength testing of potential passwords.
  - checks the password against the current password policy and rejects the password if it is weak.
  - implements a `VALIDATE_PASSWORD_STRENGTH()` SQL function that assesses the strength of potential passwords.
- To install/uninstall the `validate_password` component
  - `INSTALL COMPONENT 'file://component_validate_password';`
  - `UNINSTALL COMPONENT 'file://component_validate_password';`

```
mysql> SELECT * FROM mysql.component;
+--------------+--------------------+-----------------------------------+
| component_id | component_group_id | component_urn                     |
+--------------+--------------------+-----------------------------------+
|            1 |                  1 | file://component_validate_password |
+--------------+--------------------+-----------------------------------+
1 row in set (0.00 sec)
```

VALIDATE_PASSWORD_STRENGTH()

- > SELECT VALIDATE_PASSWORD_STRENGTH('string');

Command Prompt - mysql -u testUser -p

```
mysql> SELECT VALIDATE_PASSWORD_STRENGTH('weak');
+-----------------------------------+
| VALIDATE_PASSWORD_STRENGTH('weak') |
+-----------------------------------+
|                                25 |
+-----------------------------------+
1 row in set (0.00 sec)

mysql> SELECT VALIDATE_PASSWORD_STRENGTH('lessweak$_@123');
+----------------------------------------------+
| VALIDATE_PASSWORD_STRENGTH('lessweak$_@123') |
+----------------------------------------------+
|                                           50 |
+----------------------------------------------+
1 row in set (0.00 sec)

mysql> SELECT VALIDATE_PASSWORD_STRENGTH('N0Tweak$_@123!');
+----------------------------------------------+
| VALIDATE_PASSWORD_STRENGTH('N0Tweak$_@123!') |
+----------------------------------------------+
|                                          100 |
+----------------------------------------------+
1 row in set (0.00 sec)

mysql>
```

Password Strength Levels

- The system variable `validate_password.policy` can be set to one of the following values:
  - **`LOW`** policy tests password length only. Passwords must be at least 8 characters long.
    - To change this length, modify `validate_password.length`.
  - **`MEDIUM`** policy adds the conditions that passwords must contain at least 1 numeric character, 1 lowercase character, 1 uppercase character, and 1 special (nonalphanumeric) character.
    - To change these values, modify `validate_password.number_count`, `validate_password.mixed_case_count`, and `validate_password.special_char_count`.
  - **`STRONG`** policy adds the condition that password substrings of length 4 or longer must not match words in the dictionary file.
    - To specify the dictionary file, modify `validate_password.dictionary_file`.

Testing the Password Policy

```
mysql> ALTER USER 'testUser'@'localhost' IDENTIFIED BY 'admin123';
ERROR 1819 (HY000): Your password does not satisfy the current policy
requirements
mysql>
```

Security Guidelines

- Set Policy for good passwords!
- Further Guidelines to be discussed in next lectures:
  - Do not ever give anyone (except root accounts) access to the **"user"** table in the system database!
  - Use the GRANT and REVOKE statements to control access to database! Do not grant more privileges than necessary!
  - Do not store cleartext passwords in your database!
    - use hashing functions (e.g. SHA2) and store the hash value!

References

- Basta A., et al. (2012). Database Security. Chapter 6: Password, Profiles, Privileges, and Roles. Course Technology, Cengage Learning. ISBN: 978-1-4354-5390-6.
- https://dev.mysql.com/